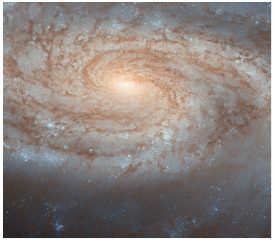
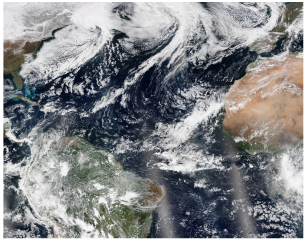
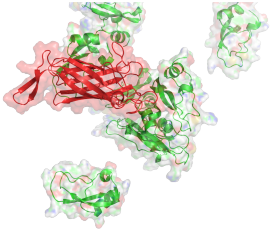
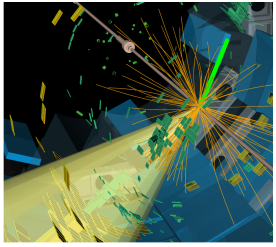





High performance scientific computing in C++

HPC C++ Course 2026

12 October – 15 October 2026 | Sandipan Mohanty | Forschungszentrum Jülich, Germany

Member of the Helmholtz Association

JÜLICH Forschungszentrum

Chapter 2

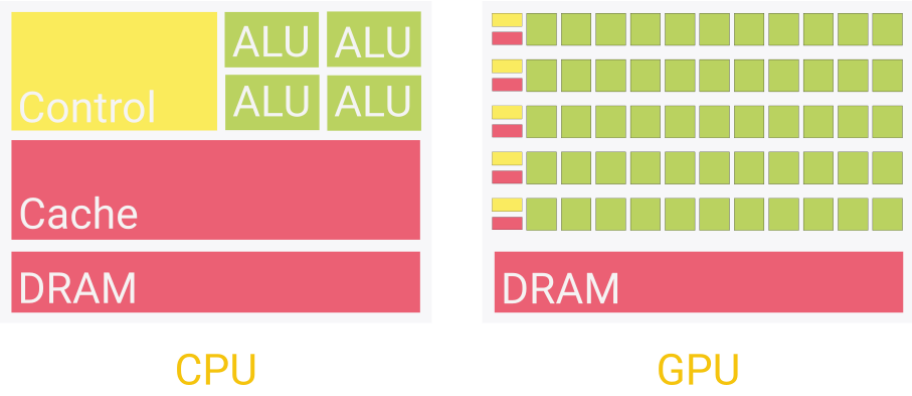
Cost of ...

JÜLICH Forschungszentrum

Data parallelism

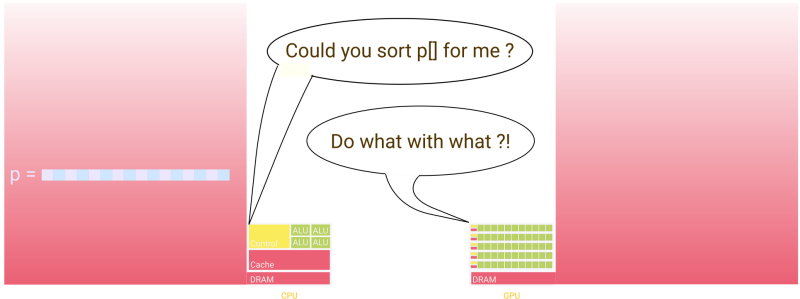
Bus/motorcycle analogy and figure stolen from GPU course/lecture slides by Andreas Herten (JSC)

Priorities



- CPU: faster clock speed, more cache, more sophisticated instructions and scheduling
- GPU: More chip area dedicated to floating point computations

A separate device



- Separate memories. GPU does not automatically know the state of any object in the memory of the CPU.
- Must transfer data.
- Must tell what to do with the data.
- Must retrieve results with another information transfer.

Can run C++ functions

- A program running on a CPU can call special functions designed to run on the GPU
- The GPU understands a different set of hardware instructions than the CPU, so any human readable function meant for the GPU must be compiled to a different kind of hardware instructions than code compiled for the CPU.
- A set of function "execution space specifiers" are provided as language extensions : `__global__`, `__device__` and `__host__`. These indicate to a CUDA aware compiler which parts to translate to the CPU language and which parts to the GPU language.
- A function running on the GPU can call other functions compiled for the GPU, leading to a call tree on the device side.

```
1  __device__ auto shuf(int id)
2  {
3      return (id + 1723) % 2000;
4  }
5  __global__
6  void gpufunc(int *ids, unsigned N)
7  {
8      // ...
9      ids[i] = shuf(ids[i]);
10     // ...
11 }
12 auto cpufunc() -> int
13 {
14     gpufunc<<<1, 100>>>(p, 3000);
15 }
16
```

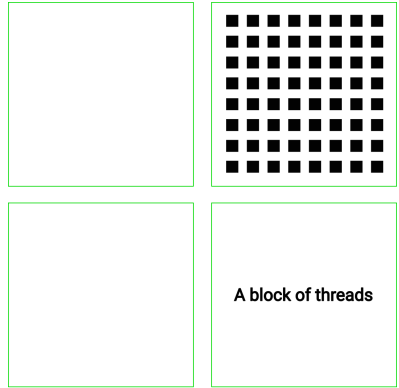

JÜLICH Forschungszentrum

Execution space specifiers

- `__device__` : the function runs on the device, and it can only be called from the device
- `__host__` : the function runs on the host, and it can only be called form the host
- `__global__`: the function is a "kernel". It runs on the device, and can be called from the host, or from device (compute capability >= 3.2)
  - Must have `void` return type
  - Can not be a member function
  - It is asynchronous : the function returns before the device performs its work
  - Must be called along with an "execution configuration" e.g., `gpufunc<<<1, 100>>>(p, 3000)`
- `__device__` and `__host__` can both be used for a function, in which case, it is compiled for both the host and the device.

Kernel call syntax

- Kernel functions are called with the `<<<GridSpec, BlockSpec>>>` notation, i.e., potentially in a large number of threads, arranged in blocks
- `BlockSpec` denotes a 3 dimensional object, 3 integers, specifying the arrangement of threads in a thread block
- `GridSpec` denotes a 3 dimensional object, 3 integers, specifying how blocks are arranged in a grid
- Each thread running a kernel function has a built in variable, `threadIdx`, specifying the position of the thread in its block, and another variable `blockIdx` to identify the block in the grid, and `blockDim` = number of threads in a block
- Overall x index: `blockIdx.x * blockDim.x + threadIdx.x` etc.



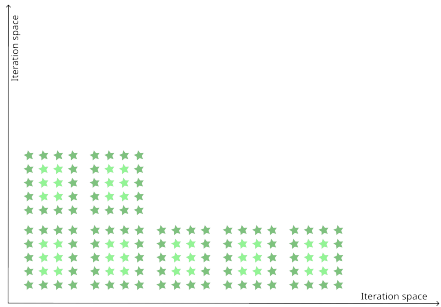
Kernel call syntax

```
1  __global__ void MatAdd(float A[N][N], float B[N][N],
2                         float C[N][N])
3  {
4      int i = blockIdx.x * blockDim.x + threadIdx.x;
5      int j = blockIdx.y * blockDim.y + threadIdx.y;
6      if (i < N && j < N)
7          C[i][j] = A[i][j] + B[i][j];
8  }
9
10 auto main() -> int
11 {
12     ...
13     // Kernel invocation
14     dim3 threadsPerBlock{16, 16};
15     dim3 numBlocks{N / threadsPerBlock.x,
16                    N / threadsPerBlock.y};
17     MatAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
18     ...
19 }
```

- The block and grid properties are often chosen to reflect properties of the problem being solved.
- In this example, the threads are organized in a 2D lattice: a natural fit for a matrix sum
- Each thread only needs to process one element!
- There is a maximum number of threads allowed in a block: a limit coming from hardware properties
- It is therefore necessary to arrange blocks into a grid

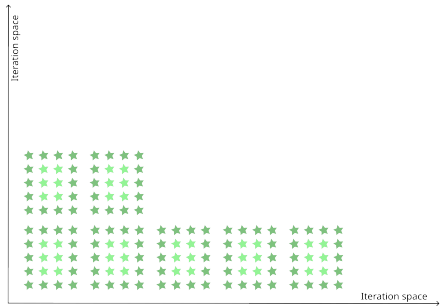
Kernel call syntax

- Remember how you had to process an array of double, 4 elements at a time and a stride of 4, when using AVX style SIMD register?



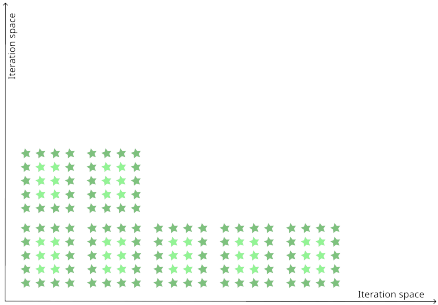
Kernel call syntax

- Remember how you had to process an array of double, 4 elements at a time and a stride of 4, when using AVX style SIMD register?
- Think of a block in CUDA as a potentially 3-dimensional stencil of tiny computations which you repeat to cover the iteration space



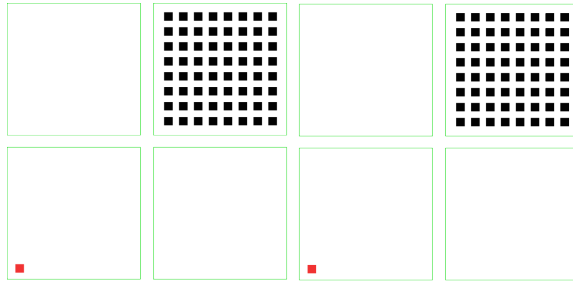
Kernel call syntax

- Remember how you had to process an array of double, 4 elements at a time and a stride of 4, when using AVX style SIMD register?
- Think of a block in CUDA as a potentially 3-dimensional stencil of tiny computations which you repeat to cover the iteration space
- We can use the grid to describe the iteration space in terms of the blocks. Depending on hardware availability, multiple blocks will run in parallel



Kernel call syntax

- Remember how you had to process an array of double, 4 elements at a time and a stride of 4, when using AVX style SIMD register?
- Think of a block in CUDA as a potentially 3-dimensional stencil of tiny computations which you repeat to cover the iteration space
- We can use the grid to describe the iteration space in terms of the blocks. Depending on hardware availability, multiple blocks will run in parallel



stride = blockDim.x * gridDim.x

- If the iteration space is much larger than the total number of GPU threads, it is sometimes helpful to do *grid stride loops* in the kernels. You have to take into account that `gridDim._ * blockDim._` GPU threads in the whole grid, which are processing lots of indexes together. That's how many indexes you would now jump over as a "stride"

Information transfer to and from the device

- Any data the kernel needs to process must be transferred using CUDA memory transfer functions
- Pointer/reference values received as input parameters in a function are interpreted on the same side of the host-device boundary

```
1  float *d_A, *d_B, *d_C;
2  auto size = N * sizeof(float);
3  cudaMalloc(&d_A, size);
4  cudaMalloc(&d_B, size);
5  cudaMalloc(&d_C, size);
6
7  cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
8  cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
```

```
1  __device__ float devData;
2  float value = 3.14f;
3  cudaMemcpyToSymbol(devData, &value, sizeof(float));
```

Information transfer to and from the device

- Any data the kernel needs to process must be transferred using CUDA memory transfer functions
- Pointer/reference values received as input parameters in a function are interpreted on the same side of the host-device boundary
- Allocations on unified memory are accessible from both the host and the device.
- Any data transfer required between the physically separate host and device memory happens automatically when using unified memory

```
1  float *u_A, *u_B, *u_C;
2  auto size = N * sizeof(float);
3  cudaMallocManaged(&u_A, size);
4  cudaMallocManaged(&u_B, size);
5  cudaMallocManaged(&u_C, size);
```

```
1  template <class T>
2  auto malloc_usm(size_t N,
3          std::optional<T> init = std::nullopt) -> T*
4  {
5      T* ans{};
6      cudaMallocManaged(&ans, N * sizeof(T));
7      if (init) {
8          for (size_t i = 0UL; i < N; ++i)
9              ans[i] = *init;
10     }
11     return ans;
12 }
```


JÜLICH Forschungszentrum

Device side memory hierarchy and memory space specifiers

- "Local memory" -> per thread memory
- "Shared memory" -> private to a block, but shared among the threads inside a block
- "Global memory" -> visible from all threads in all blocks
- "Constant memory" -> also in the device space, and cached in the constant cache

- Memory address specifier `__device__` declares a variable which lives on the device
- `__constant__` declares a variable to be stored in constant cache
- `__shared__` : variable for the shared memory inside a block, and has the lifetime of the block
- `__managed__`: A variable declared with managed storage specifier can be accessed from both the host and the device, We can determine its address, and it can be read/written from both the host and the device. Since the host and device memories are physically separate, this behaviour is achieved by transferring memory implicitly

Example

```
1  __global__ void mul(const double *A, const double *B, double *C, size_t N) {
2      auto i = threadIdx.x + blockIdx.x * blockDim.x;
3      auto j = threadIdx.y + blockIdx.y * blockDim.y;
4      double res{};
5      if (i < N && j < N)
6          for (size_t k = 0ul; k < N; ++k)
7              res += A[N * i + k] * B[N * k +j];
8      C[N*i + j] = res;
9  }
10 auto main(int argc, char *argv[]) -> int {
11     const unsigned N = (argc > 1) ? std::stoul(argv[1]) : 2048u;
12     auto a = malloc_usm<double>(N * N);
13     auto b = malloc_usm<double>(N * N);
14     auto c = malloc_usm<double>(N * N);
15     for (size_t i = 0UL; i < N * N; ++i) { a[i] = b[i] = 1.1; }
16     auto t0 = std::chrono::high_resolution_clock::now();
17     dim3 ThreadsPerBlock{16, 16};
18     dim3 NumBlocks{N / ThreadsPerBlock.x, N / ThreadsPerBlock.y};
19     mul<<<NumBlocks, ThreadsPerBlock>>>(a, b, c, N);
20     cudaDeviceSynchronize();
```

This is simply a syntax demonstration! Not a particularly clever implementation!

Compiling CUDA code

- With `nvcc` :

```
nvcc [--extended-lambda] [-std=__] source.cu
```

- With `clang++` :

```
clang++ [-std=__] source.cc --cuda-gpu-arch=____ \
    -I /path/to/CUDA/include \
    -L /path/to/CUDA/lib64 -lcudart_static -ldl -lrt -lpthread
```

In the classroom setup on JURECA, the GPU architecture parameter should be given as `sm_80`. The `-L` options may be skipped. But currently, the include needs this: `$(dirname $(which nvcc) )/../include/cccl` (If using the module CUDA) and `$(dirname $(which nvcc) )/../include-stdpar` (If using the module nvidia-compilers)

CUDA and C++

- Except in some ancient versions, CUDA is parsed by the rules of the C++ language. Many perfectly valid code in C, e.g., using `class`, `new`, `using` etc. as variable names can not be part of CUDA programs
- Valid C++ code, can often not be used, for a variety of reasons:
  - Generally, the NVIDIA implementation of newer language features arrives a few years after standardization
  - Some language features may have to be modified for use in the context of GPUs
  - CUDA 13 supports most of C++20. Our working environment is based on CUDA 13.0, or, if you load the module `ml nvidia-compilers`, NVidia HPC C++ compiler version 25.9
  - No modules. A few other smaller restrictions
  - `consteval` functions defined for the host can be invoked in the device code context: after all those are supposed to be immediately evaluated by the compiler!
- Execution space specifiers, execution configuration etc. are language extensions
- This sometimes means additional rules are necessary before a new language feature can be used with CUDA. E.g., how do we make a lambda function `__device__` ? Should `__host__` etc. be considered parts of the functions signature or not ? `nvcc` and `clang++` disagree !

NVIDIA Thrust

```
1  #include <thrust/host_vector.h>
2  #include <thrust/device_vector.h>
3  #include <thrust/generate.h>
4  #include <thrust/sort.h>
5  #include <thrust/copy.h>
6  #include <cstdlib>
7  using namespace thrust;
8  auto main() -> int
9  {
10     // generate 32 M random numbers on
11     // the host
12     host_vector<int> h_vec(32 << 20);
13     generate(h_vec.begin(), h_vec.end(), rand);
14
15     // transfer data to the device
16     device_vector<int> d_vec = h_vec;
17     sort(d_vec.begin(), d_vec.end());
18     // transfer data back to the host
19     copy(d_vec.begin(), d_vec.end(), h_vec.begin());
20 }
```

- Template library like STL or TBB for CUDA
- Elegant high level syntax (STL like iterator interface for algorithms, clever use of operator overloading ...) to clearly express the intent of the programmer
- The compiler translates the stated intents to efficient code for the GPU
- Primarily NVIDIA GPUs

NVIDIA Thrust

```
1  #include <thrust/host_vector.h>
2  #include <thrust/device_vector.h>
3  #include <thrust/generate.h>
4  #include <thrust/sort.h>
5  #include <thrust/copy.h>
6  #include <cstdlib>
7  using namespace thrust;
8  auto main() -> int
9  {
10     // generate 32 M random numbers on
11     // the host
12     host_vector<int> h_vec(32 << 20);
13     generate(h_vec.begin(), h_vec.end(), rand);
14
15     // transfer data to the device
16     device_vector<int> d_vec = h_vec;
17     sort(d_vec.begin(), d_vec.end());
18     // transfer data back to the host
19     copy(d_vec.begin(), d_vec.end(), h_vec.begin());
20 }
```

- Example: `thrust::host_vector` and `thrust::device_vector` use the assignment operator to transfer data between the CPU and the GPU

NVIDIA Thrust

```cpp
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/generate.h>
#include <thrust/sort.h>
#include <thrust/copy.h>
#include <cstdlib>
using namespace thrust;
auto main() -> int
{
    // generate 32 M random numbers on
    // the host
    host_vector<int> h_vec(32 << 20);
    generate(h_vec.begin(), h_vec.end(), rand);

    // transfer data to the device
    device_vector<int> d_vec = h_vec;
    sort(d_vec.begin(), d_vec.end());
    // transfer data back to the host
    copy(d_vec.begin(), d_vec.end(), h_vec.begin());
}
```

- Example: `thrust::host_vector` and `thrust::device_vector` use the assignment operator to transfer data between the CPU and the GPU
- Thrust algorithms like `thrust::sort` have syntax like STL algorithms

JÜLICH Forschungszentrum

NVIDIA Thrust

```
1  #include <thrust/host_vector.h>
2  #include <thrust/device_vector.h>
3  #include <thrust/generate.h>
4  #include <thrust/sort.h>
5  #include <thrust/copy.h>
6  #include <cstdlib>
7  using namespace thrust;
8  auto main() -> int
9  {
10     // generate 32 M random numbers on
11     // the host
12     host_vector<int> h_vec(32 << 20);
13     generate(h_vec.begin(), h_vec.end(), rand);
14
15     // transfer data to the device
16     device_vector<int> d_vec = h_vec;
17     sort(d_vec.begin(), d_vec.end());
18     // transfer data back to the host
19     copy(d_vec.begin(), d_vec.end(), h_vec.begin());
20 }
```

- Example: `thrust::host_vector` and `thrust::device_vector` use the assignment operator to transfer data between the CPU and the GPU
- Thrust algorithms like `thrust::sort` have syntax like STL algorithms
- Many data parallel general operations have their own algorithms: `transform`, `reduce`, `inclusive_scan`

Host and device vectors

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <iostream>
auto main() -> int
{
    thrust::host_vector<int> H(4);
    for (int i = 0; i < 4; ++i) H[i] = i;
    // resize H
    H.resize(2);
    std::cout << "H now has size "
        << H.size() << "\n";
    // Copy host_vector H to
    // device_vector D
    thrust::device_vector<int> D = H;
    // elements of D can be modified
    D[0] = 99;
    D[1] = 88;
    // print contents of D
    for(int i = 0; i < D.size(); ++i)
        std::cout << "D[" << i << "] = "
                  << D[i] << "\n";
}
```

- Containers `host_vector` and `device_vector` are designed similar to `std::vector`, but, do not have initializer list constructors or new member functions of `std::vector` like `emplace_back`

Host and device vectors

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <iostream>
auto main() -> int
{
    thrust::host_vector<int> H(4);
    for (int i = 0; i < 4; ++i) H[i] = i;
    // resize H
    H.resize(2);
    std::cout << "H now has size "
        << H.size() << "\n";
    // Copy host_vector H to
    // device_vector D
    thrust::device_vector<int> D = H;
    // elements of D can be modified
    D[0] = 99;
    D[1] = 88;
    // print contents of D
    for(int i = 0; i < D.size(); ++i)
        std::cout << "D[" << i << "] = "
                  << D[i] << "\n";
}
```

- Containers `host_vector` and `device_vector` are designed similar to `std::vector`, but, do not have initializer list constructors or new member functions of `std::vector` like `emplace_back`
- The overloaded assignment operators can copy data across devices

Other initialization options

```
1  // initialize all ten integers to 1
2  thrust::device_vector<int> D(10, 1);
3  // set the first seven elements to 9
4  thrust::fill(D.begin(), D.begin() + 7, 9);
5  // initialize a host_vector with
6  // the first five elements of D
7  thrust::host_vector<int> H(D.begin(), D.begin() + 5);
8  // set elements of H to 0, 1, 2, ...
9  thrust::sequence(H.begin(), H.end());
```

- Many algorithms to provide initial values, to serve different purposes.
- There is also `thrust::generate` which can call a functional for every element of the vector
- The type of the iterators tell the compiler which version of the respective algorithms to use. No run-time overhead

Exercise 3.1:

The example programs `examples/thrust0.cu` and `examples/thrust1.cu` contain the thrust code in the previous slides. Run them on JURECA using the following steps:

- Load the NVidia HPC module: `ml nvidia-compilers`
- Compile using the `nvcc` or the `nvc++` compiler: `nvcc thrust0.cu`
- Build with `clang` (when `nvidia-compilers` is loaded)

```
clang++ -std=c++23 thrust0.cu -I $(dirname $(which nvcc) )/../include-stdpar\
    --cuda-gpu-arch=sm_80 -lcudart_static -ldl -lrt -lpthread
```

- Try changing the file name to `thrust0.cc` and compiling

Thrust algorithms

```
1   device_vector<int> X(10), Y(10), Z(10);
2   // initialize X to 0, 1, 2, 3, ....
3   sequence(X.begin(), X.end());
4   // compute Y = -X
5   thrust::transform(X.begin(), X.end(),
6      Y.begin(), thrust::negate<int>());
7   // fill Z with twos
8   thrust::fill(Z.begin(), Z.end(), 2);
9   // compute Y = X mod 2
10  thrust::transform(X.begin(), X.end(),
11                Z.begin(), Y.begin(),
12                thrust::modulus<int>());
13  // replace all the ones in Y with 10
14  thrust::replace(Y.begin(), Y.end(), 1, 10);
15  // print Y
16  thrust::copy(Y.begin(), Y.end(),
17  std::ostream_iterator<int>(cout, "\n"));
```

- Host and device versions
- A set of elementary functionals are available in `thrust/functional.h`
- Notice the copy from a device vector to the ostream iterator!

Universal vectors

```cpp
// examples/thrust_usm.cc
#include <thrust/universal_vector.h>
#include <thrust/sort.h>
#include <iostream>
auto main() -> int
{
    thrust::universal_vector<int> h_vec(1 << 22);
    std::cout << "Filling host vector with random numbers\n";
    thrust::generate(thrust::host, h_vec.begin(), h_vec.end(), rand);
    std::cout << "Done.\n";

    std::cout << "Sorting vector on device\n";
    thrust::sort(thrust::device, h_vec.begin(), h_vec.end());
    std::cout << "Done.\n";
}
```

- `thrust::universal_vector` is similar to `thrust::host_vector` and `thrust::device_vector`, but uses unified memory for storage

Universal vectors

```cpp
// examples/thrust_usm.cc
#include <thrust/universal_vector.h>
#include <thrust/sort.h>
#include <iostream>
auto main() -> int
{
    thrust::universal_vector<int> h_vec(1 << 22);
    std::cout << "Filling host vector with random numbers\n";
    thrust::generate(thrust::host, h_vec.begin(), h_vec.end(), rand);
    std::cout << "Done.\n";

    std::cout << "Sorting vector on device\n";
    thrust::sort(thrust::device, h_vec.begin(), h_vec.end());
    std::cout << "Done.\n";
}
```

- `thrust::universal_vector` is similar to `thrust::host_vector` and `thrust::device_vector`, but uses unified memory for storage
- Data does not need to be moved explicitly between host and device

Universal vectors

```cpp
// examples/thrust_usm.cc
#include <thrust/universal_vector.h>
#include <thrust/sort.h>
#include <iostream>
auto main() -> int
{
    thrust::universal_vector<int> h_vec(1 << 22);
    std::cout << "Filling host vector with random numbers\n";
    thrust::generate(thrust::host, h_vec.begin(), h_vec.end(), rand);
    std::cout << "Done.\n";

    std::cout << "Sorting vector on device\n";
    thrust::sort(thrust::device, h_vec.begin(), h_vec.end());
    std::cout << "Done.\n";
}
```

- `thrust::universal_vector` is similar to `thrust::host_vector` and `thrust::device_vector`, but uses unified memory for storage
- Data does not need to be moved explicitly between host and device
- Algorithms need to be told whether they are meant for host or device explicitly

Custom functionals for transforms

```
1  struct saxpy_functor {
2    const float a;
3    saxpy_functor(float _a) : a(_a) {}
4    __host__ __device__
5      auto operator()(const float& x,
6                      const float& y) const -> float {
7        return a * x + y;
8      }
9  };
10 void saxpy_fast(float A,
11       const thrust::device_vector<float>& X,
12       thrust::device_vector<float>& Y)
13 {
14   // Y <- A * X + Y
15   thrust::transform(X.begin(), X.end(),
16                     Y.begin(), Y.begin(),
17                     saxpy_functor(A));
18 }
```

- When pre-defined operations in `thrust/functional.h` do not suffice, we can write our own function objects
- The overloaded `operator()` must be marked with `__host__ __device__`

JÜLICH Forschungszentrum

Custom functionals using placehoders

- For very simple operations, custom functionals can be generated inline using the `thrust::placehoders` namespace.

```
1  void saxpy_fast(float A,
2        thrust::device_vector<float>& X,
3        thrust::device_vector<float>& Y)
4  {
5      // Y <- A * X + Y
6      thrust::transform(X.begin(), X.end(),
7                        Y.begin(), Y.begin(),
8                        (A * _1 + _2));
9  }
```

- `_1`, `_2` ... are placehoders
- Expressions involving placeholders yield a functional mapping its arguments sequencially to `_1`, `_2` ...

Custom functionals using lambda functions

```
1  void saxpy_fast(float A,
2        thrust::device_vector<float>& X,
3        thrust::device_vector<float>& Y)
4  {
5      // Y <- A * X + Y
6      thrust::transform(X.begin(), X.end(), Y.begin(), Y.begin(),
7                        [A] __host__ __device__ (double x, double y) {
8          return A * x + y;
9      });
10 }
```

```
nv++  saxpy0.cu
```

- Note where we mark the lambda function to be for the host and device

Exercise 3.2: Placeholders and lambda functions

The example `examples/saxpy0.cu` shows how to use the placehoders with `thrust` algorithms for simple inline functionality. There is also a commented out version of the same thing done using a lambda function. The placeholder version is more compact, but the lambda version can have multiple statements, like a normal function.

Exercise 3.3: Mandelbrot set

The Mandelbrot set is the set of complex numbers c for which the function $f(z) = z^2 + c$ does not diverge when iterated from $z = 0$. An image representing the set can be created by generating the sequence $z_n = z_{n-1}^2 + c$ for each pixel in the image, by treating the x and y values of the pixel as the real and imaginary components of c. The sequence can be taken to have diverged if the magnitude of z exceeds 2. The program `exercises/mandelbrot_cpu.cc` does it, using the standard C++ library. A modified version using `thrust`, `mandelbrot_gpu.cu` is also present. Build `nvcc` and `clang++` and run. Figure out the code differences and why they are needed.

STDPAR: standard C++ for GPUs

- NVC++, the NVIDIA HPC SDK C++ compiler
- No `<<< >>>`, no `__device__` etc. Just plain C++ written with STL algorithms
- `std::execution::par` regions automatically translated into GPU code!
- There are restrictions, but they will likely be fewer and fewer in the future

```
std::transform_reduce(std::execution::par, R2.begin(),
    R2.end(), S12.begin(), 0., std::plus<double>{},
    [](auto r2, auto s12){
        return Vexv(r2, s12);
    });
```

```
nvc++ -O3 -std=c++23 -stdpar exvol.cc -o exvol.nv
```

STDPAR: standard C++ for GPUs

- Only inline functions or function templates. `nvc++` selects functions for GPU execution on its own, and that only works if it can see the definitions
- CUDA Managed Memory (`-gpu=mem:managed`) or CUDA Unified Memory (`-gpu=mem:unified`) for all data movement between CPU and GPU. In hte managed mode, only heap allocated objects in CPU code compiled by `nvc++ -stdpar` can be accessed. Stack and global storage not accessible. Even heap allocations from portions of CPU code not compiled by `nvc++ -stdpar` are not visible.
- Pointers dereferenced in the parallel algorithms must point to heap locations. References used must be of heap objects.
- Lambda captures by references can often entail pointer dereferencing for stack entities, which should not occur in parallel algorithm regions
- No function pointers: functions are compiled for CPU and GPU. Pointer can only point to one. Inside GPU code, there will then be the danger of accessing a pointer to a function with CPU code. Pass function objects or lambdas as arguments to the algorithms instead.
- Only random access iterators
- `catch` clauses in GPU code ignored. Fine inside CPU code.
- On Grace-Hopper Superchip systems, like JUPITER at the JSC, and stack variables, global or static objects, and heap allocated data, all become available from both CPU and GPU.

Exercise 3.4:

The programs `stdpardemo0.cc` – `stdpardemo2.cc` are simple short programs using parallel algorithms. The second one is a slighly modified version of the `exvol.cc` program we used in connection with SIMD programming. Compile them with `nvc++` and run them on a GPU node on JURECA.

Exercise 3.5:

The program `jacobi.cc` is in your examples folder. Identify the part which can be parallelized using STL parallel algorithms, and do the necessary code changs. It can be compiled for the GPU using `nvc++`, without any code changes. Try this new way of CPU/GPU programming where the exact same code runs on both!

SYCL

- General model for heterogeneous computing: CPU, GPU, FPGA...
- Can create queues for different devices
- Single source, portable, but not necessarily performance portable
- Open standard from Khronos
- Ref: Data Parallel C++

```
1  template <class T>
2  using usm_alloc_t = sycl::usm_allocator<T,
3      sycl::usm::alloc::shared>;
4  template <class T>
5  using my_vector = std::vector<T, usm_alloc_t<T>>;
6  auto main() -> int
7  {
8      using std::numbers::pi;
9      constexpr auto N = 1UL << 20UL;
10     sycl::queue q;
11
12     usm_alloc_t<double> usmq;
13     myvector<double> v{N, usmq};
14
15     auto* vraw = v.get();
16
17     q.submit([&](sycl::handler& h) {
18         h.parallel_for(N, [=](auto i) {
19             v[i] = 2 * pi * i / N;
20         });
21     }).wait();
22 }
```

SYCL

- Use `sycl::usm_allocator` along with `std::vector` to create USM vectors to be used in kernels
- Alternatively USM `sycl::malloc` and `sycl::free` calls in an RAII helper class, similar to a unique pointer
- Need at least one queue, e.g., `sycl::queue q;`. For more control, `sycl::queue q_gpu{gpu_selector{}};`
- Submit tasks to the queues, containing parallel algorithm calls.
- The task functional submitted should accept a `sycl::handler&`, a command group handler, as the argument

```
1  template <class T>
2  using usm_alloc_t = sycl::usm_allocator<T,
3      sycl::usm::alloc::shared>;
4  template <class T>
5  using my_vector = std::vector<T, usm_alloc_t<T>>;
```

```
ml AdaptiveCPP/25.10.0
acpp -std=c++20 -O3 prog.cc -o prog.ex
```

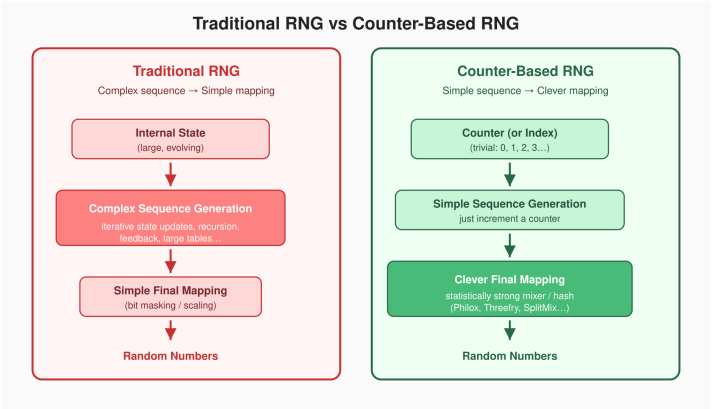
Run it on a CPU or a GPU!

JÜLICH Forschungszentrum

Exercise 3.6:

Example programs `examples/conv_sycl_usm.cc` and `examples/gblur_sycl.cc` demonstrate iteration over a 1D and 2D space using SYCL. They both perform a calculation similar to a convolution as a demo. The later one also demonstrates how to receive information about the hardware. Build it using the AdaptiveCPP compiler as shown and try to run the generated executable with the `batch_run` command for a CPU run and the `batch_run_gpu` command for a GPU run.

Counter based random number generators



- Left (Traditional): Heavy complexity lives in the sequence generation step; the final mapping is simple.
- Right (Counter-based): Sequence generation is trivial (just a counter); the statistical quality comes from a clever final mapping.

Exercise 3.7:

The example `mock_random_sycl.cc` uses a self-contained mock up of a random number generator which can be used in parallel in millions of threads simultaneously, suitable for a GPU. Proper random number generators for GPUs exist, like CUDA curand, OneMKL RNG etc. But for something without any outside dependencies, and only needing to look {textitsomewhat random for tests, you could use something like this. It uses the counter based approach with the split mix mixing step to produce a random sequence in parallel.